

Експерт. Інтеграція API ШІ в проєкті

№ уроку: 3 Курс: FrontEnd.AI Skills

Ціль уроку

Використання API LLM у фронтенд-проєкті (через fetch / axios / SDK).
Використання LLM локально (через Ollama / локальні рушії, приклади виклику з JS).
Знайомство з JS-екосистемою ML (TensorFlow.js, ONNX Runtime Web, ml5.js)

Зміст уроку:

1. Що таке API LLM?
2. Які основні способи взаємодії з API LLM?
3. Як використовувати API LLM з JavaScript (fetch, SDK, Node.js)?
4. Які задачі ШІ API може вирішувати?
5. Що таке TensorFlow.js / ONNX Runtime / ml5.js?
6. Чим відрізняється JS-ML від LLM?
7. Як використовувати LLM локально і як викликати локальну LLM з JS?
8. Online vs Offline: переваги й недоліки.
9. Що таке токен в контексті ШІ?

Резюме

Урок присвячений тому, як інтегрувати можливості великих мовних моделей у власні продукти через API LLM та як це поєднати з JavaScript і браузерними ML-інструментами. Студенти знайомляться з поняттям API LLM як програмного інтерфейсу, що дозволяє “розмовляти” з моделлю через HTTP-запити: надсилати промпти, отримувати відповіді в JSON і використовувати їх у чат-ботах, Q&A-сервісах, генерації текстів, коду, SQL-запитів, автоматизації e-mail і модерації контенту. Розбираються ключові параметри запиту (model, messages, temperature, max_tokens тощо), типові способи взаємодії (REST, SDK, streaming, no-code інтеграції) та практичні приклади виклику API з JavaScript/Node.js.

Окремо розглядаються сценарії, які вирішуються через AI-API в ширшому сенсі: обробка тексту (NLP, переклад, аналіз настроїв, витяг сутностей), робота з зображеннями (розпізнавання, генерація, OCR), аудіо (speech-to-text, text-to-speech), рекомендаційні системи, безпека й модерація. Студенти дізнаються, як ШІ допомагає в розробці: генерувати й переписувати тексти, підсумовувати документи, створювати код та автотести, будувати чат-ботів.

Урок також знайомить з підходами до запуску моделей локально (Ollama, LM Studio, формати ONNX/GGUF) та браузерними ML-бібліотеками (TensorFlow.js, ONNX Runtime Web, ml5.js), пояснюючи різницю між роботою через хмарний API та локальним inference: приватність vs простота, якість моделей vs вимоги до ресурсів. Наприкінці вводиться поняття токена як базової одиниці тексту, від якої залежать ліміти контексту та вартість запитів, і обговорюються практичні плюси та мінуси використання платних і безкоштовних AI-сервісів для реальних фронтенд-проєктів.

Питання для самоперевірки

- Що таке API LLM?
- Назвіть способи взаємодії з API.

- Які задачі можна вирішити за допомогою API?
- Що таке TensorFlow.js / ONNX / ml5.js?
- У чому відмінність між JS-ML (TensorFlow.js) і LLM?
- Як запустити LLM локально?
- Що таке токен?

Домашнє завдання

1. Перейдіть на сайт <https://ollama.ai/>
2. Скачайте та встановіть клієнт Ollama
3. Запустіть Ollama сервер (зазвичай запускається автоматично)
4. У командному рядку запустіть "ollama pull phi3" і скачайте LLM модель phi3
5. Створіть веб-інтерфейс (HTML/CSS/JS) для генерації тексту через Ollama API
6. Використайте ендпоінт <http://localhost:11434/api/generate> для REST запити
7. Реалізуйте потокове отримання відповіді (streaming) від моделі phi3
8. Додайте можливість змінювати модель через інтерфейс
9. Додайте кнопки для генерації, зупинки та очищення результату

Додаткові ресурси

- Документація OpenAI: <https://platform.openai.com/docs>
- Ollama: <https://ollama.com/>
- Groq Console: <https://console.groq.com/>
- TensorFlow.js: <https://www.tensorflow.org/js>
- ONNX Runtime Web: <https://onnxruntime.ai/docs/how-to/wasm.html>
- ml5.js: <https://ml5js.org/>
- Hugging Face: <https://huggingface.co/>
- Документація про токени (OpenAI): <https://platform.openai.com/tokenizer> (або відповідний розділ на сайті провайдера)